



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Spring, Year: 2025), B.Sc. in CSE (Day)

Lab Report: 04

Course Title: Data Mining Lab
Course Code: CSE 436
Section: 212_D2

Lab Manual 05

Implementation of the Apriori Algorithm for Frequent Itemset Mining

Student Details

Name	ID
Md. Almahmud	212002142

Submission Date : 30-04-2025
Course Teacher's Name : Md. Atikuzzaman

Lab Report Status

Marks :Signature.....

Comments :Date :.....

1. TITLE OF THE LAB EXPERIMENT:

Implementation of the Apriori algorithm for frequent itemset mining.
Use “*Market_Basket_Optimisation.csv*” Dataset (Attached).

2. OBJECTIVES/AIM:

- To implement the Apriori algorithm in Python for identifying frequent itemsets in transactional data.

3. MOTIVATION:

- In retail and e-commerce industries, understanding purchasing patterns is vital for marketing strategies like product bundling, recommendation systems, and shelf organization.
- The Apriori algorithm helps in mining frequent item sets and discovering association rules, which reveal how items are commonly purchased together.

4. PROCEDURE:

- Load the dataset “*Market_Basket_Optimisation.csv*”.
- Preprocess the data into a list of transactions.
- Apply the Apriori algorithm using appropriate support and confidence thresholds.
- Extract frequent itemsets and association rules.
- Interpret results to identify strong associations between items.

5. CODE:

```
!pip install mlxtend

import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=FutureWarning,
module="seaborn")
warnings.filterwarnings("ignore", category=Warning)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", None)

df = pd.read_csv("/content/Market_Basket_Optimisation.csv",
header=None, index_col=None, names=[f"Item_{i}" for i in range(1,
21)])
df.head()

txns = df.fillna("").values.tolist()
txns = [[item for item in txn if item != ''] for txn in txns]
txns = [[item.strip() for item in txn] for txn in txns]

ids = [i + 1 for i in range(len(txns))]
data = []
```

```

for i, txn in enumerate(txns):
    data.extend([{'TID': ids[i], 'Item': item} for item in txn])

df_txn = pd.DataFrame(data)
df_txn.head(25)

from mlxtend.preprocessing import TransactionEncoder
te = TransactionEncoder()
te_array = te.fit(txns).transform(txns)
te_columns = te.columns_
df1 = pd.DataFrame(te_array, columns=te.columns_)
df1.head()

top_items = df_txn['Item'].value_counts().reset_index()
df_top_items = pd.DataFrame(top_items)
df_top_items.columns = ['Item', 'Count']
total_transactions = len(df)
df_top_items['% Count'] = (df_top_items['Count']*100 /
total_transactions).round(2)
df_top_items.style.background_gradient(cmap='Blues')

from mlxtend.frequent_patterns import apriori
min_support_threshold = 0.04
frequent_itemsets = apriori(df1, min_support=min_support_threshold,
use_colnames=True)
frequent_itemsets

from mlxtend.frequent_patterns import association_rules
confidence_threshold = 0.3
rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=confidence_threshold)
sorted_rules = rules.sort_values(['confidence', 'support', 'lift'],
ascending=[False, False, False])
sorted_rules

```

6. OUTPUT:

```

from mlxtend.frequent_patterns import apriori
min_support_threshold = 0.04
frequent_itemsets = apriori(df1, min_support=min_support_threshold,
use_colnames=True)
frequent_itemsets

```

	support	itemsets
0	0.087188	(burgers)
1	0.081056	(cake)
2	0.046794	(champagne)
3	0.059992	(chicken)
4	0.163845	(chocolate)
5	0.080389	(cookies)
6	0.051060	(cooking oil)
7	0.179709	(eggs)
8	0.079323	(escalope)
9	0.170911	(french fries)
10	0.043061	(fresh bread)

```

from mlxtend.frequent_patterns import association_rules
confidence_threshold = 0.3
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=confidence_threshold)
sorted_rules = rules.sort_values(['confidence', 'support', 'lift'], ascending=[False, False, False])
sorted_rules

```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift
1	(ground beef)	(mineral water)	0.098254	0.238368	0.040928	0.416554	1.747522
2	(milk)	(mineral water)	0.129583	0.238368	0.047994	0.370370	1.553774
3	(spaghetti)	(mineral water)	0.174110	0.238368	0.059725	0.343032	1.439085
0	(chocolate)	(mineral water)	0.163845	0.238368	0.052660	0.321400	1.348332

```
from mlxtend.frequent_patterns import apriori
min_support_threshold = 0.04
frequent_itemsets = apriori(df1, min_support=min_support_threshold)
frequent_itemsets
```

	support	itemsets
0	0.087188	(burgers)
1	0.081056	(cake)
2	0.046794	(champagne)
3	0.059992	(chicken)
4	0.163845	(chocolate)
5	0.080389	(cookies)
6	0.051060	(cooking oil)
7	0.179709	(eggs)
8	0.079323	(escalope)
9	0.170911	(french fries)
10	0.043061	(fresh bread)

25	0.050527	(soup)
26	0.174110	(spaghetti)
27	0.068391	(tomatoes)
28	0.062525	(turkey)
29	0.058526	(whole wheat rice)
30	0.052660	(mineral water, chocolate)
31	0.050927	(mineral water, eggs)
32	0.040928	(mineral water, ground beef)
33	0.047994	(milk, mineral water)
34	0.059725	(spaghetti, mineral water)

7. DISCUSSION:

The dataset includes records of items bought in each transaction, where every row represents a customer's shopping cart. To analyze this, the data was converted into a list of transactions. This format allowed the use of the Apriori algorithm, which is designed to find frequently purchased item sets. A minimum support threshold was set to ignore items that appear too rarely, helping focus on the most common and meaningful item combinations. This helps reduce noise and speeds up the analysis process.

From the frequent itemsets, association rules were generated using two key metrics: support (how often an item or item set appears) and confidence (how often the rule has been found to be true). For example, rules like "If a customer buys bread and butter, they also buy jam" or "If milk is bought, cereal is often bought too" were discovered. These insights can help improve cross-selling, product bundling, and store layout optimization in retail settings.